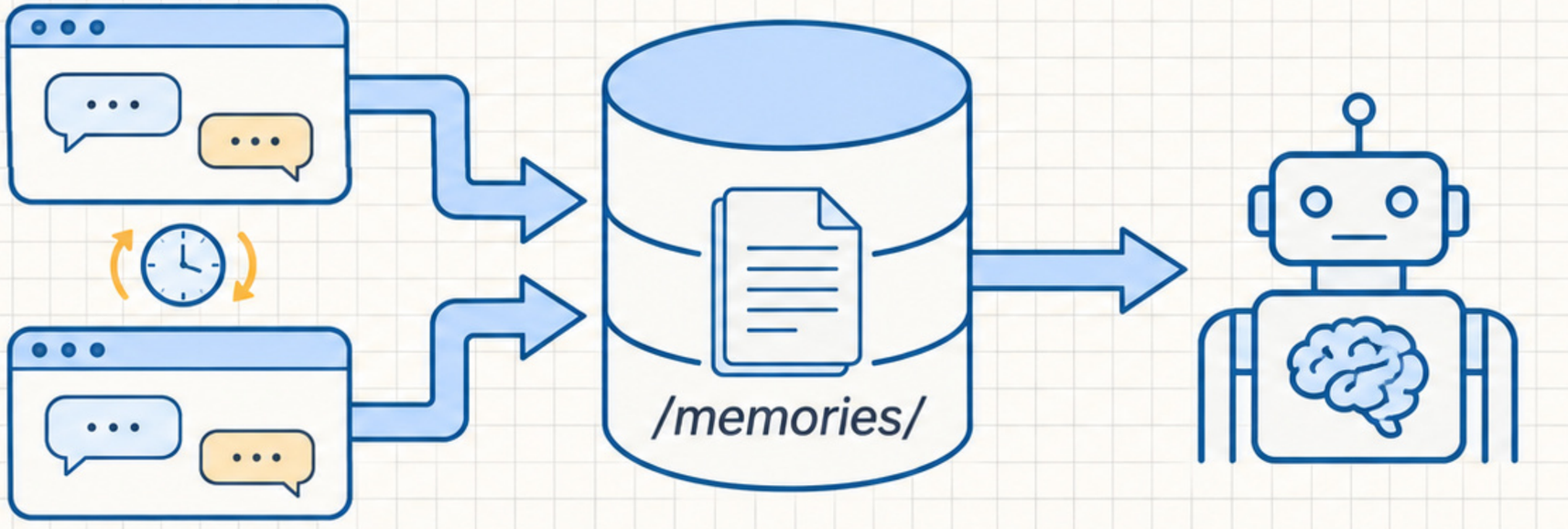
Deep Agents 实战

第 8 章：长期记忆

让 Agent 在不同对话间复用信息



线程状态　　共享存储　　路径路由

两种记忆：隔离不等于删除



短期：按 thread_id 隔离

- Checkpointer 可保存并恢复旧线程
- 新线程看不到旧线程 State 文件



长期：不同线程访问同一 Store namespace

- 文件可跨线程共享
- 重启后是否保留取决于 Store

对话结束不会自动删除旧线程；MemorySaver 与 InMemoryStore 都是进程内存。

Memory：先准备，再加载，再更新

1 准备：配置 Backend、Store，预置文件



2 加载：memory=[文件路径] 读取已有内容



3 更新：约定目标路径，调用文件工具，再核对保存结果



memory= 不会自动创建缺失文件，也不保证模型已经写入。

skills= 接受目录；

memory= 接受文件。

作用域：namespace 决定共享范围

作用域	namespace	共享范围
 Agent 级	`(assistant_id,)`	同一 Agent **共享知识**
 用户级	`(user_id,)`	按**可信** user_id 分区
 组织级	`(org_id,)`	同一组织共享策略，通常限制为**只读**



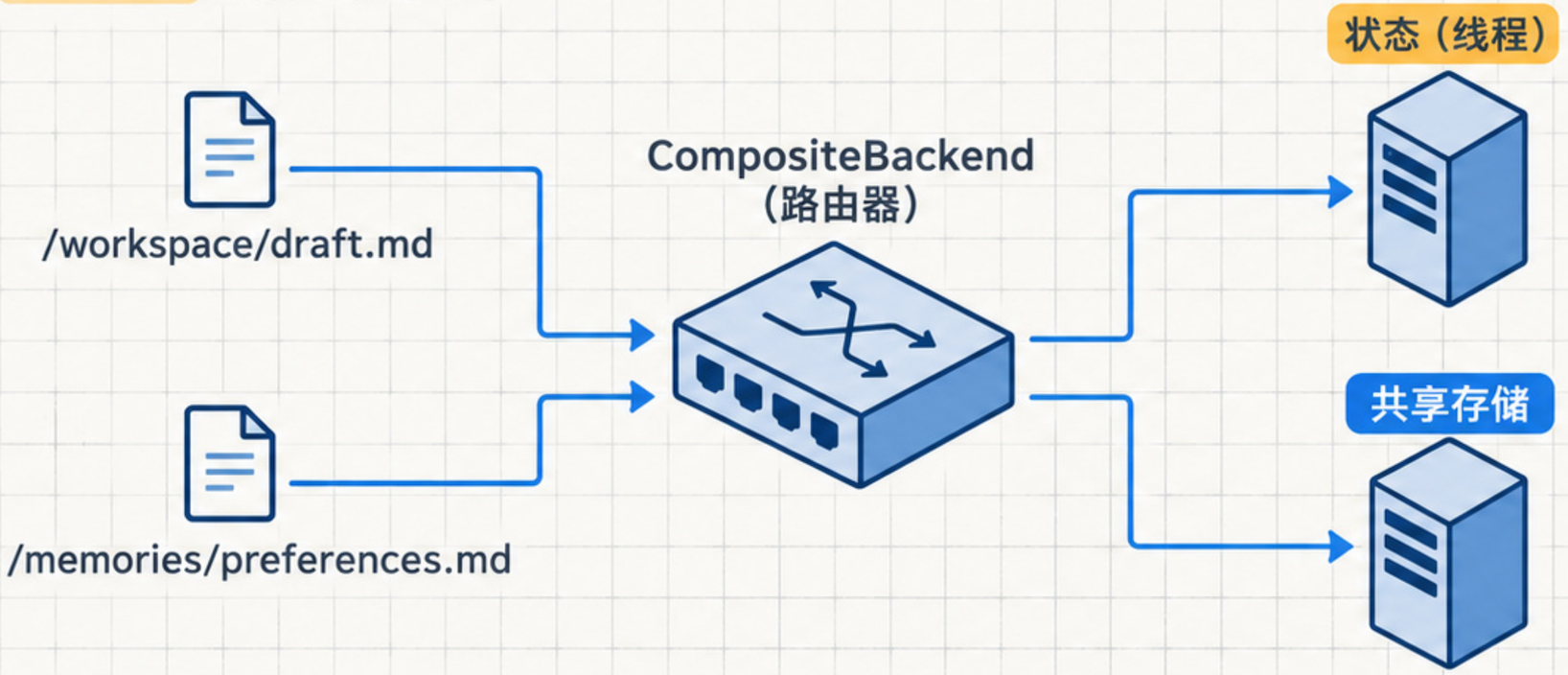
身份由**服务端**或**可信 context** 提供；
不要直接相信模型给出的 user_id。



本地 server_info 可能为空；
生产中缺少身份应**拒绝**，而非共用默认用户。

CompositeBackend：路径决定后端

Router 路由示意图



StateBackend

线程状态，可由 Checkpointer 保存

StoreBackend

跨线程共享相同 namespace

细节 | 挂载 /memories/ 后，Store key 是 /preferences.md

路由只分配存储位置，不会自动赋予持久性或只读权限。

跨线程验证：不能只看“已记住”

1 初始化文件



Store + create_file_data

2 对话 A



读取并 edit_file 更新偏好

3 应用核验



检查工具结果与 Store 内容

4 新 thread_id



检查模型请求中的记忆及回答

两次调用复用同一 Store 与 namespace；

InMemoryStore 重建后数据不在。

加载生命周期：同线程不等于重新读取

首次加载



从配置文件读取到 memory_contents

同线程恢复



已有 memory_contents 时跳过重新下载

外部文件更新



不保证立即进入已缓存的系统提示词

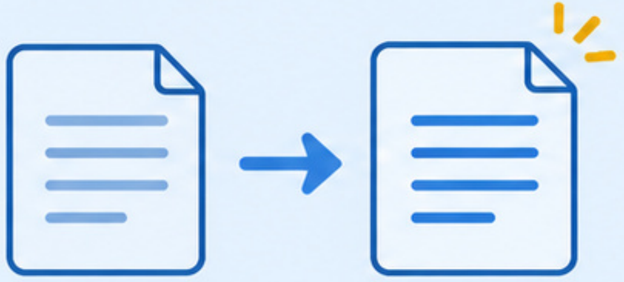


用全新 thread_id 检查新加载；必要时通过文件工具读取当前内容。

以 deepagents 0.7.22 MemoryMiddleware 源码核对。

写入策略：更新不等于覆盖

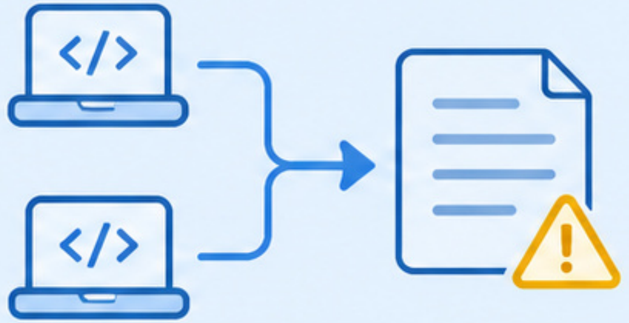
write_file：完整覆盖已有内容



edit_file：匹配片段后局部替换



并发写：同一文件可能出现覆盖冲突



先读后改、检查结果；按主题拆分或写事件记录，再串行整合。

提示：提示词是约定，不是强制权限；敏感共享记忆需审批与写入限制。

记忆系统的六个维度

	持续时间	线程内 / 跨线程；实际保留**由存储决定**
	信息类型	经历 / 事实 / 任务方法
	作用域	用户 / Agent / 组织
	更新策略	对话内 / 后台整合
	读取方式	memory 加载 / 文件工具按需读取
	权限控制	读写 / 受控只读

⚠ 这些是设计选择，不是配置一个 `memory=` 就全部自动实现。

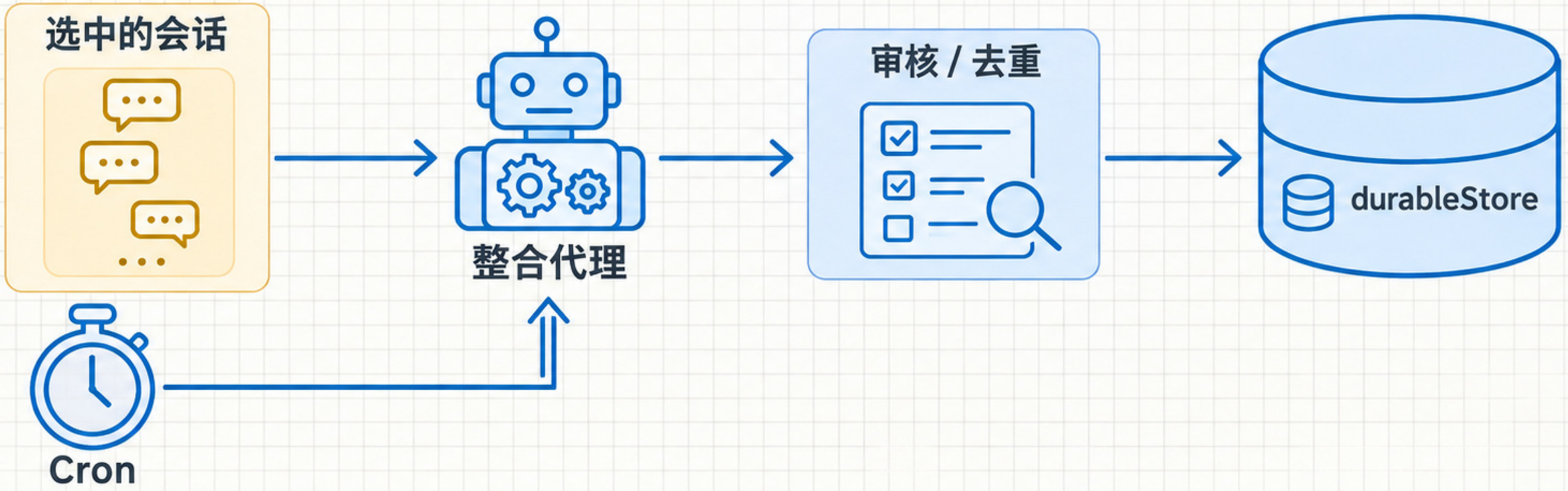
后台整合：需要独立工程闭环

配置真实部署与定时触发

可信身份过滤与必要分页

复用同一 Backend、Store、namespace

审核后串行写入并记录结果



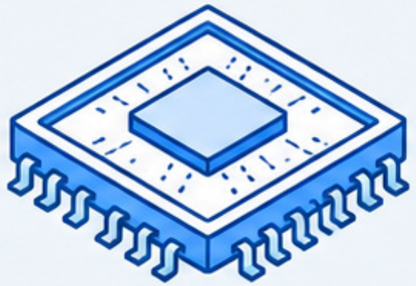
调度间隔需覆盖回溯窗口；还要处理失败重试、去重与遗漏。

从开发到生产：存储要真正落地

跨线程可用 ≠ 跨重启持久。

开发

InMemoryStore



- 进程内，重启丢失

自建生产

PostgresStore



- 初始化表，管理连接生命周期

托管部署

由部署平台提供 Store



- 确认实际配置

预置文件用 `create_file_data`；先检查是否存在，**避免重跑覆盖**旧记忆。

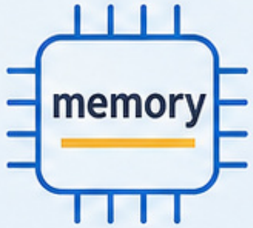
本章回顾：把“记住”做成可验证结果



配置存储
与已有文件



按可信身份划分
namespace



memory
负责加载



更新后核对
Store 与 trace



新线程
验证复用



Checkpointer
保存线程状态。



Store
承载跨线程共享。

下一章：Human-in-the-Loop，为敏感操作加入人工审批。